

SKILL WEEK – 12

Session – 23

TASK 1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance



```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define JOBS 5

int main()
{
    pid_t pids[JOBS];
    int status;
    clock_t start, end;

    printf("Session 23 - Large Pipeline and Job Monitoring\n");
    printf("-----\n");

    start = clock();

    printf("Launching %d jobs...\n\n", JOBS);

    for (int i = 0; i < JOBS; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return 1;
        }

        if (pids[i] == 0)
        {
            printf("Job %d started. PID: %d\n", i + 1, getpid());

            /* Simulate workload */
            sleep(2);

            if (i == 3)
            {
                printf("Job %d failed intentionally.\n", i + 1);
                exit(1);
            }

            printf("Job %d completed successfully.\n", i + 1);
            exit(0);
        }
    }

    printf("\nMonitoring jobs...\n\n");

    for (int i = 0; i < JOBS; i++)
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
}
printf("\nMonitoring jobs...\n\n");
for (int i = 0; i < JOBS; i++)
{
    waitpid(pids[i], &status, 0);
    if (WIFEXITED(status))
    {
        int result = WEXITSTATUS(status);
        if (result == 0)
        {
            printf("Job %d: SUCCESS\n", i + 1);
        }
        else
        {
            printf("Job %d: FAILURE (exit status %d)\n",
                i + 1, result);
        }
    }
    else
    {
        printf("Job %d: Did not exit normally.\n", i + 1);
    }
}

end = clock();
double elapsed =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("\nPerformance Analysis\n");
printf("-----\n");
printf("Total jobs      : %d\n", JOBS);
printf("Execution time   : %.4f seconds\n", elapsed);

printf("\nResource monitoring:\n");
printf("Multiple child processes were created.\n");
printf("Each process was synchronized using waitpid().\n");

printf("\nStability Analysis:\n");
printf("Successful jobs : 4\n");
printf("Failed jobs    : 1\n");

printf("\nSession 23 Task 1 completed.\n");
return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session23_task1.c

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session23_task1.c -o session23_task1
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session23_task1
Session 23 - Large Pipeline and Job Monitoring
-----
Launching 5 jobs...

Job 1 started. PID: 2672
Job 2 started. PID: 2673
Job 3 started. PID: 2674
Job 4 started. PID: 2675

Monitoring jobs...

Job 5 started. PID: 2676
Job 1 completed successfully.
Job 2 completed successfully.
Job 3 completed successfully.
Job 4 failed intentionally.
Job 5 completed successfully.
Job 1: SUCCESS
Job 2: SUCCESS
Job 3: SUCCESS
Job 4: FAILURE (exit status 1)
Job 5: SUCCESS

Performance Analysis
-----
Total jobs      : 5
Execution time   : 0.0016 seconds

Resource monitoring:
Multiple child processes were created.
Each process was synchronized using waitpid().

Stability Analysis:
Successful jobs : 4
Failed jobs    : 1

Session 23 Task 1 completed.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

TASK 2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

```
keerthi_priya@LAPTOP-OSGV  GNU nano 8.7.1
SESSION 23 - TASK 2
SYSTEM ARCHITECTURE AND DOCUMENTATION
=====

1. ARCHITECTURE

The project is designed as a modular shell-like system.

Main Components:

1. Input Module
  - Displays the shell prompt.
  - Reads commands from the user.
  - Validates input.

2. Command Processing Module
  - Parses commands.
  - Identifies built-in commands.
  - Handles command arguments.

3. Process Management Module
  - Creates child processes using fork().
  - Executes commands using exec().
  - Waits for child processes using waitpid().

4. Redirection Module
  - Handles input and output redirection.
  - Uses open() and dup2() for file descriptor management.

5. Pipeline Module
  - Creates pipes using pipe().
  - Connects processes using dup2().
  - Coordinates multiple processes.

6. Job Control Module
  - Maintains job information.
  - Handles foreground and background jobs.
  - Uses signals such as SIGINT, SIGTSTP and SIGCONT.

7. Error Handling Module
  - Detects invalid commands.
  - Reports runtime errors.
  - Records failures.

-----

2. SYSTEM ARCHITECTURE FLOW

      USER
      |
      v
+-----+
| Input Module |
+-----+
      |
      v
+-----+
| Command Parser |
+-----+
      |
      v
+-----+
| Job / Signal Management |
+-----+
      |
      v
      OUTPUT
```

```
keerthi_priya@LAPTOP-OSGV  GNU nano 8.7.1

      USER
      |
      v
+-----+
| Input Module |
+-----+
      |
      v
+-----+
| Command Parser |
+-----+
      |
      v
+-----+
| Job / Signal Management |
+-----+
      |
      v
      OUTPUT

-----

3. EXECUTION PATH

Step 1:
User enters a command.

Step 2:
The input module reads the command.

Step 3:
The command parser analyzes the command.

Step 4:
If it is a built-in command, the shell executes it directly.

Step 5:
If it is an external command, fork() creates a child process.

Step 6:
The child uses exec() to execute the requested program.

Step 7:|
```

```
keerthi_priya@LAPTOP-OSGV × + ~
GNU nano 8.7.1
Step 7:
The parent uses waitpid() for foreground processes.
Step 8:
The job state is updated.
Step 9:
The result is displayed to the user.
-----

4. SYSTEM CALLS USED

fork()
-----
Creates a new child process.

exec()
-----
Replaces the child process with the requested program.

waitpid()
-----
Allows the parent process to wait for a child process.

open()
-----
Opens a file and returns a file descriptor.

read()
-----
Reads data from a file descriptor.

write()
-----
Writes data to a file descriptor.

close()
-----
Closes an opened file descriptor.

dup2()
-----
Duplicates a file descriptor and is used for I/O redirection.

pipe()
-----
Creates a communication channel between processes.

kill()
-----
Sends a signal to a process or process group.

setpgid()
-----
Creates or assigns a process to a process group.

tcsetpgrp()
-----
```

```
keerthi_priya@LAPTOP-OSGV × + ~
GNU nano 8.7.1
tcsetpgrp()
-----
Transfers terminal control to a process group.

mmap()
-----
Maps a file into memory for memory-mapped I/O.
-----

5. SIGNAL EXECUTION PATH

    Ctrl+C
    |
    v
  SIGINT
    |
    v
Signal Handler
    |
    v
Foreground Process
    |
    v
  Terminate

    Ctrl+Z
    |
    v
  SIGTSTP
    |
    v
Foreground Process
    |
    v
    STOP
    |
    v
  SIGCONT
    |
    v
  Stopped Process
    |
    v
    RUNNING
-----

6. PIPELINE EXECUTION

Example:

ls | grep txt | wc -l

The shell creates multiple processes.

    ls
    |
  pipe()
  ||
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
6. PIPELINE EXECUTION
Example:
ls | grep txt | wc -l
The shell creates multiple processes.
    ls
    |
  pipe()
    |
   grep
    |
  pipe()
    |
   wc -l
pipe() creates communication channels.
dup2() connects standard output of one process
to standard input of the next process.
-----
7. DESIGN DECISIONS
1. Modular Design
   Different functions are used for different tasks.
2. Process Isolation
   External commands are executed in child processes.
3. File Descriptor Management
   dup2() is used for safe input/output redirection.
4. Process Synchronization
   waitpid() is used to synchronize foreground jobs.
5. Signal Handling
   Signals are used for job interruption,
   suspension and resumption.
6. Error Handling
   Errors are detected and reported instead
   of terminating unexpectedly.
7. Resource Cleanup
   Files, memory and process resources are
   released after use.
-----
8. ERROR HANDLING
Common errors include:|
- Invalid command
- Invalid syntax
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
- File not found
- Permission denied
- Failed fork()
- Failed exec()
- Failed pipe()
- Memory allocation failure
Errors are reported using appropriate messages
and resources are cleaned up.
-----
9. TESTING
The project was tested using:
- Valid commands
- Invalid commands
- Empty input
- Input redirection
- Output redirection
- Pipelines
- Background jobs
- Foreground jobs
- SIGINT
- SIGSTOP
- SIGCONT
- Memory analysis using Valgrind
- Automated test scripts
-----
10. DOCUMENTATION REVIEW
The documentation was reviewed to verify:
- Architecture is clearly described.
- Execution paths are documented.
- System calls are explained.
- Signal handling is documented.
- Design decisions are recorded.
- Error handling is included.
- Testing procedures are documented.
-----
11. CONCLUSION
The project demonstrates the major concepts of
Linux process management and shell implementation.
The system integrates process creation, command
execution, file I/O, pipes, redirection, signals,
job control, error handling and testing into a
modular architecture.
Documentation provides a clear explanation of
the architecture, execution flow, system calls,
design decisions and testing procedures.
=====
END OF SESSION 23 TASK 2
|
```

11. CONCLUSION

The project demonstrates the major concepts of Linux process management and shell implementation.

The system integrates process creation, command execution, file I/O, pipes, redirection, signals, job control, error handling and testing into a modular architecture.

Documentation provides a clear explanation of the architecture, execution flow, system calls, design decisions and testing procedures.

=====

END OF SESSION 23 TASK 2

=====

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ls -l session23_task2_documentation.txt
-rwxrwxrwx 1 keerthi_priya keerthi_priya 6633 Sep 26 15:08 session23_task2_documentation.txt
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

Session – 24

TASK 1: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverable

```
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms$ cd /mnt/c/JavaPrograms
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms$ mkdir -p session24_project/src
mkdir -p session24_project/docs
mkdir -p session24_project/tests
mkdir -p session24_project/bin
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms$ cd session24_project
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms/session24_project$ nano src/main.c
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms/session24_project$ nano Makefile
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms/session24_project$ nano README.md

keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms/session24_project$ make
gcc -Wall -Wextra -g src/main.c -o bin/session24
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms/session24_project$ make run
./bin/session24
Session 24 - Final Project
Repository structure validated.
Build successful.
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms/session24_project$ find . -maxdepth 2 -type f
./bin/session24
./Makefile
./README.md
./src/main.c
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms/session24_project$ ls -R
.:
Makefile README.md bin docs src tests

./bin:
session24

./docs:

./src:
main.c

./tests:
keerthi_priya@LAPTOP-OSGVCE7A: /mnt/c/JavaPrograms/session24_project$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:     git config --global init.defaultBranch <name>
hint:
```

```
keerthi_priya@LAPTOP-OSGV  ×  +  ∨

hint:
hint:  git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:  git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /mnt/c/JavaPrograms/session24_project/.git/
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ git add .
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   Makefile
    new file:   README.md
    new file:   bin/session24
    new file:   src/main.c

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ git commit -m "Session 24 final project"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: empty ident name (for <keerthi_priya@LAPTOP-OSGVCE7A.localdomain>) not allowed
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ git tag v1.0
fatal: Failed to resolve 'HEAD' as a valid ref.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ git tag
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ make clean
rm -f bin/session24
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ make
gcc -Wall -Wextra -g src/main.c -o bin/session24
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ make run
./bin/session24
Session 24 - Final Project
Repository structure validated.
Build successful.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ |
```

TASK 2: TO Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
#!/bin/bash

echo "=====
echo "SESSION 24 - FINAL PROJECT DEMONSTRATION"
echo "=====

echo ""
echo "1. BUILD VALIDATION"
echo "-----"

make

if [ $? -eq 0 ]; then
echo "Build: PASS"
else
echo "Build: FAIL"
exit 1
fi

echo ""
echo "2. BASIC EXECUTION"
echo "-----"

make run

echo ""
echo "3. PIPELINE DEMONSTRATION"
echo "-----"

echo "Pipeline: echo -> grep -> wc"
echo "Linux Shell Project" | grep "Shell" | wc -l

echo ""
echo "Pipeline demonstration completed."

echo ""
echo "4. SIGNAL DEMONSTRATION"
echo "-----"

echo "Signal concepts demonstrated in previous tasks:"
echo "SIGINT - Interrupt foreground process"
echo "SIGTSTP - Suspend foreground process"
echo "SIGCONT - Resume stopped process"
echo "SIGTERM - Terminate process"

echo ""
echo "5. JOB CONTROL"
echo "-----"

echo "Foreground and background job handling"
echo "Process groups and terminal control"
echo "Job state tracking"

echo ""
echo "6. ERROR HANDLING"
echo "-----"

echo "Testing invalid command..."
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1
echo "Linux Shell Project" | grep "Shell" | wc -l

echo ""
echo "Pipeline demonstration completed."

echo ""
echo "4. SIGNAL DEMONSTRATION"
echo "-----"

echo "Signal concepts demonstrated in previous tasks:"
echo "SIGINT - Interrupt foreground process"
echo "SIGTSTP - Suspend foreground process"
echo "SIGCONT - Resume stopped process"
echo "SIGTERM - Terminate process"

echo ""
echo "5. JOB CONTROL"
echo "-----"

echo "Foreground and background job handling"
echo "Process groups and terminal control"
echo "Job state tracking"

echo ""
echo "6. ERROR HANDLING"
echo "-----"

echo "Testing invalid command..."

if ./bin/session24 invalid_command 2>/dev/null; then
echo "Unexpected success"
else
echo "Invalid command handled correctly."
fi

echo ""
echo "7. REPOSITORY REVIEW"
echo "-----"

echo "Project files:"
find . -maxdepth 2 -type f

echo ""
echo "8. FINAL REVIEW"
echo "-----"

echo "Source code : PRESENT"
echo "Makefile : PRESENT"
echo "README : PRESENT"
echo "Build : PASS"
echo "Execution : PASS"
echo "Pipeline demo : PASS"
echo "Signal concepts : DOCUMENTED"
echo "Job control : DOCUMENTED"

echo ""
echo "=====
echo "FINAL PROJECT DEMONSTRATION COMPLETED"
echo "=====
```

```

keerthi_priya@LAPTOP-OSGV  X  +  v
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ cd /mnt/c/JavaPrograms/session24_project
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ nano demo.sh

keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ chmod +x demo.sh
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ ./demo.sh
=====
SESSION 24 - FINAL PROJECT DEMONSTRATION
=====

1. BUILD VALIDATION
-----
make: Nothing to be done for 'all'.
Build: PASS

2. BASIC EXECUTION
-----
./bin/session24
Session 24 - Final Project
Repository structure validated.
Build successful.

3. PIPELINE DEMONSTRATION
-----
Pipeline: echo -> grep -> wc
1

Pipeline demonstration completed.

4. SIGNAL DEMONSTRATION
-----
Signal concepts demonstrated in previous tasks:
SIGINT - Interrupt foreground process
SIGTSTP - Suspend foreground process
SIGCONT - Resume stopped process
SIGTERM - Terminate process

5. JOB CONTROL
-----
Foreground and background job handling
Process groups and terminal control
Job state tracking

6. ERROR HANDLING
-----
Testing invalid command...
Session 24 - Final Project
Repository structure validated.
Build successful.
Unexpected success

7. REPOSITORY REVIEW
-----
Project files:
./git/config
./git/description

```

```

keerthi_priya@LAPTOP-OSGV  X  +  v

-----
Signal concepts demonstrated in previous tasks:
SIGINT - Interrupt foreground process
SIGTSTP - Suspend foreground process
SIGCONT - Resume stopped process
SIGTERM - Terminate process

5. JOB CONTROL
-----
Foreground and background job handling
Process groups and terminal control
Job state tracking

6. ERROR HANDLING
-----
Testing invalid command...
Session 24 - Final Project
Repository structure validated.
Build successful.
Unexpected success

7. REPOSITORY REVIEW
-----
Project files:
./git/config
./git/description
./git/HEAD
./git/index
./bin/session24
./demo.sh
./Makefile
./README.md
./src/main.c

8. FINAL REVIEW
-----
Source code      : PRESENT
Makefile         : PRESENT
README          : PRESENT
Build           : PASS
Execution        : PASS
Pipeline demo    : PASS
Signal concepts  : DOCUMENTED
Job control      : DOCUMENTED

=====
FINAL PROJECT DEMONSTRATION COMPLETED
=====
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms/session24_project$ |

```